

CS – 432: Databases

Assignment – II

The GitHub Link to access the project content is given below:

https://github.com/Pallav-Biyala/CS432_Track1_Assignment_2_Submission

Module – A: Implementation of B+ Trees

The link to the YouTube video containing explanation of B+ Tree is:

<https://youtu.be/80A03CMBiss?si=7ft3SJVrdwf-PRVr>

🌀 Introduction to B+ Trees:

A B+ tree is a balanced tree data structure commonly used in database systems and file systems to efficiently manage and retrieve large amounts of data. It is a variant of Binary tree just all the data is stored in leaf nodes.

One of the main advantages of using B+ tree in our databases systems is that all major operations: insert, search, delete takes **$O(\log n)$** time. Additionally, leaf nodes in a B+ Tree are linked together like linked list which enables efficient range queries and sequential access, which are essential operations in database management system.

Due to these properties, B+ tree is widely used as index structures in databases. Hence, in this module we implemented a standard **B+ tree based indexing system** in Python Programming Language to support efficient data operations such as insertion, deletion, search and range queries.

Additionally, we performed **performance benchmarking** and **visualization** to compare the B+ Tree implementation with a brute-force approach and to analyze its efficiency.

So let's go to the methodologies:

🌀 Methodology of B+ Trees:

The core component of this module is the implementation of B+ Tree to efficiently store and retrieve data records. The methodology followed focuses on designing B+ tree such that it supports efficient insertion, deletion, search and range queries.

➔ B+ Tree Structure:

The B+ tree we implemented consists of:

1. Internal nodes where, each node contains key and child pointers which guide the search process on basis of Binary Search (BS) Property.
2. All actual data records are stored in the leaf nodes.
3. Tree remains balanced, i.e., all leaf nodes are at same level
4. Leaf nodes are linked sequentially like linked lists allowing efficient traversal across range queries.

One of the most important thing in B+ Tree is **order**. The order of B+ tree determines the maximum number of children each internal node can have, which actually affects the height of the tree and efficiency of operations.

➔ Node organization:

The tree, as discussed consists of two types of nodes:

- **Internal Nodes:** Contains only key and pointers to child node. They are used only to guide the way to leaf nodes as per BS property.
- **Leaf Nodes:** Stores the {key, value} pairs which are actually the real data we store. It also contains pointer to the next leaf node, enabling sequential access for range queries.

So to implement the node structure, we create a class of object Node which looks like:

```
class BPlusTreeNode:
    def __init__(self, leaf = False):
        self.leaf = leaf # tells if current node is leaf or not
        self.keys = [] # stores all keys in current node
        self.children = [] # stores all children of the current node
        self.values = [] # stores all records
        self.next = None # used for leaf to maintain linked list of leaf nodes
        self.parent = None # parent pointer
```

Hence, it contains:

- Keys[] → To store all keys
- Values[] → To store values (actual records)
- Leaf → to know if the current node is leaf or not
- Children[] → This is the children pointer that points to children
- Next → for leaf nodes to sequentially connect
- Parent → to store the parent of current node. Used for insertion and deletion during overflow and underflow

→ B+ Tree Operations:

So, let's understand the B+ Tree Operations:

□ Insertion:

When inserting a new key into B+ Tree:

1. So we can firstly find the leaf node where the key is supposed to get inserted by just traversing from root and using BS Property. (we created findLeaf() function to do this)
2. Then once we get the node, we insert key into the keys list as well as value into the value list of node while maintaining sorted order.
3. If the node exceeds the maximum capacity as per order of B+ Tree (**max m-1 keys**), i.e., **overflows**, we split the nodes:
 - Node is divided into two nodes
 - A key is promoted to the parent node (generally first child of right splitted)
4. This process propagates upward if parent node overflows.

Hence, this is how we maintain balance of tree during insertion and insert keys.

□ Deletion:

So to delete a node from B+ Tree:

1. So we can firstly find the leaf node where the key is present using findLeaf() function. Then it just removes it from the list of leaf node.
2. If the removing causes the node to fall below minimum number of keys as per order ($\text{ceil}((\text{order}-1)/2)$), i.e., **underflow**, then we can have three cases:
 - **Borrow from left:** If the left node has enough keys that it can donate its rightmost key without getting underflow, then the underflow node takes one key from left node.
 - **Borrow from right:** If in case left node is on verge of underflow, then underflow node asks right child for a key. If the right node has enough keys that it can donate its leftmost key without getting underflow, then the underflow takes one key from right node.
 - **Merge:** If borrowing is not possible, then the underflow node merges with the left node and forms a new node and then parent nodes also get updated if BS property is disturbed.

Hence, these operations maintain the structural constraints of B+ Tree while keeping it balanced.

❑ Search:

Search is again simply like find the leaf node where key given is supposed to be there using findLeaf() and then brute force through node. If it has key, return it otherwise none.

❑ Range Query:

So since all records are just in leaf node, we just need to search the start key and reach that leaf node. Then we just need to traverse the leaf node from the given start key and then continue traversing linked list until we reach the end key or a key which is just less than end key.

By this way, we can simply perform range query.

🌀 Functionality:

The system we developed in this module provides a simplified database-like environment built on top of a B+ Tree indexing structure. The implementation supports several core functionalities that allow efficient storage, retrieval, and management of records. But here, let's focus more on B+ trees:

(We have also implemented table.py that using B+ Tree we created as indexing. Also we have db_manager.py that's creates database and adds table to it using table.py that is using B+ Tree. This is included in test files as well as GitHub but here we would focus more on B+ Tree only.)

❑ B+ Tree:

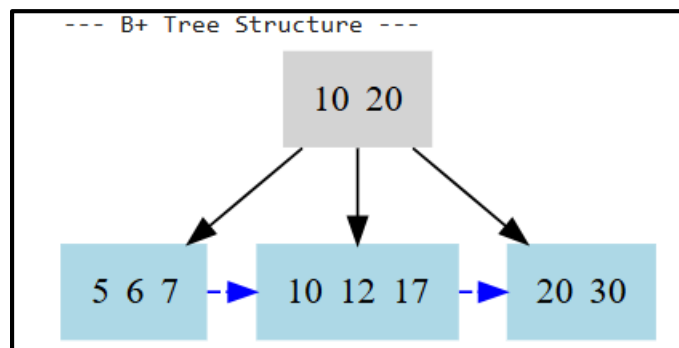
So the B+ Tree we have implemented forms core indexing of system. It supports following operations:

1. Insertion:

So, we have inserted couple of values in B+ tree and it looks like:

Keys Inserted: [10, 20, 5, 6, 12, 30, 7, 17]

The tree looks like: (Visualization through Graphviz.)



This image is taken from the "test_bplustree.ipynb" that we used as test file for bplustree. The code file is available in GitHub link at top of page.

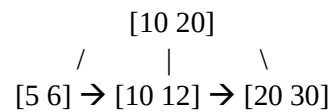
So, if we try to explain: (order is 4, so max 3 keys are allowed)

- At first, we can simply add keys 10, 20 and 5 → [5 10 20]
- As we add key 6, the node becomes → [5 6 10 20]. Since it overflows the node splits into [5 6] and [10 20]
- And the key [10] becomes the parent node. Hence

$$\begin{array}{c} [10] \\ / \quad \backslash \\ [5 \ 6] \rightarrow [10 \ 20] \end{array}$$

- Then we add 12 and hence being >10, it adds to [10 20] becoming [10 12 20]

- Then we add 30 and hence being >10 , it adds to $[10\ 12\ 20]$ becoming $[10\ 12\ 20\ 30]$ and hence its overflows as key exceeds max (3). So it splits into $[10\ 12]$ and $[20\ 30]$ and the first key of right child $[20]$ becomes parent appending to parent $[10]$. Hence root becomes $[10\ 20]$
So our tree becomes:

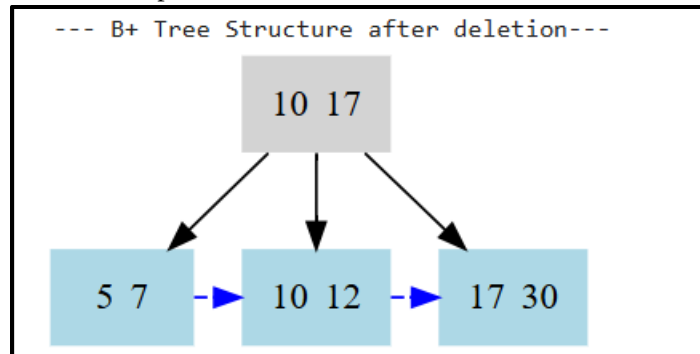


- And now we can see that, left of 10 has all keys <10 . Middle node has all keys between 10 to 20 and right has all keys greater than 20
- Now, as we add 7 since being <10 , it adds to $[5\ 6]$ becoming $[5\ 6\ 7]$
- And then we add 17 and hence being in between 10 and 20, $[10\ 20]$ becomes $[10\ 17\ 20]$

And hence we get the tree same as what we saw.

2. Deletion:

So our B+ Tree also supports deletion operation. So if we delete 6 and 20, then the tree we created becomes:



This image is taken from the “`test_bplustree.ipynb`” that we used as test file for bplustree. The code file is available in GitHub link at top of page.

So, let's understand what happened:

- As we deleted 6, the node became $[5\ 7]$ which has 2 keys and since a node should have at least 2 keys for order 4, it doesn't underflow
- As we remove 20, the right node becomes $[30]$ and hence underflows. Hence, it borrows one key (rightmost one) from left node and became $[17\ 30]$.
But now the BS property disturbed as right of 20, we had $[17\ 30]$ (17 is less than 20 and hence can't be there). So hence, parent also modified and picked removed 20 and picked first key of right node 17 becoming $[10\ 17]$

That's how deletion happens.

3. Search:

Our B+ tree also supports, search operation. Let's demonstrate it.

We searched for 6 and 15 (before deletion) and it returned:

```

--- Search Tests ---
Search 6: Value6
Search 15 (not present): None

```

This image is taken from the “`test_bplustree.ipynb`” that we used as test file for bplustree. The code file is available in GitHub link at top of page.

Then after deleting 6 and 20, we searched again for 6 and 20 and it returned:

```
--- Deleting keys ---  
After deleting 6 and 20:  
Search 6: None  
Search 20: None
```

This image is taken from the “[test_bplustree.ipynb](#)” that we used as test file for bplustree. The code file is available in GitHub link at top of page

Hence, our B+ Tree also does search

4. Range Query:

So our B+ Tree can also perform range query.

We wanted to print the records from 1 to 10 and hence, used it. Our B+ Tree returned:

```
--- Final Range Query 1 to 10 ---  
(5, 'Value5')  
(7, 'Value7')  
(10, 'Value10')
```

This image is taken from the “[test_bplustree.ipynb](#)” that we used as test file for bplustree. The code file is available in GitHub link at top of page

5. Update Query:

Our B+ tree also supports update query where we can update value of any key in leaf nodes to desired value.

→ Performance Analyzer and Visualizer:

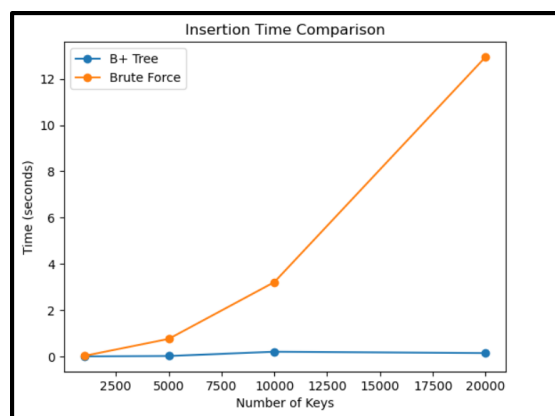
The performance of the B+ Tree was evaluated by comparing it with the brute force database. So for this we created the performance analyzer class that included measure time of various operations like Insert, delete, search, range query and random operations.

So, we ran the benchmark code on both data structures over dataset size of [1000, 5000, 10000, 20000] and then plotted the graph for time taken vs size.

B+ tree being $O(\log n)$ for operations had very small logarithmic growth as n increased as compared to brute force which has $O(n)$.

Let's see an example:

Insertion: B+ tree had $O(\log n)$ while brute force has $O(n)$ and hence b+ tree took very small amount of time while brute force took more time for same size.



This graph is taken from the “[report.ipynb](#)”. The report file is available in GitHub link at top of page

Similar results were obtained and graphs were noted in the report.ipynb. We can refer the explanation and results from there along with proper explanation. Hence in summary, B+ tree is faster and better than brute force in terms of speed of executing operations.

🌀 Approach:

The problem that we wanted to solve was the storage issue on disks. Data structures like B tree, AVL take lot of space and are very slow in operations. Hence, to tackle it, we implemented a B+ Tree.

1. So we started with the creation of B+ tree node which included keys, values, children pointer, parent pointer, isleaf and next pointer.
2. After creation of node, we studied structure of B+ Tree and then learnt how insertion, deletion, search takes place in the B+ Tree.
3. Then we implemented all operations required in python code.
4. Then, to compare B+ tree and understand why it is used, we created performance analyzer module that calculates time for different operations of different data structures. Then we used it to compare B+ tree and brute force based on time and space complexity.
5. After that, we used Graphviz to visualize the whole B+ Tree using graph nodes and links.
6. Then we prepared the report and added all in GitHub link.

Hence, this was all about B+ Trees.

Module – B: Local API Development, RBAC, and Database Optimization

The link to the video containing explanation is:

<https://drive.google.com/file/d/1HSxMbX55cCmvgTl1kV7NbYKjCu1MXkZ/view?usp=sharing>

The link to the github repo is: https://github.com/Pallav-Biyala/CS432_Track1_Assignment_2_Submission

🌀 Introduction:

This project focuses on designing and implementing a full-stack database-driven web application with proper authentication, role-based access control (RBAC), and CRUD operations.

The system allows users to interact with structured data through a web interface while ensuring secure and controlled access based on user roles such as Admin and Regular User.

🌀 Objectives:

- Build a full-stack application using modern technologies
- Implement a relational database system
- Enable CRUD operations (Create, Read, Update, Delete)
- Apply Role-Based Access Control (RBAC)
- Integrate frontend and backend via APIs
- Maintain logs for system actions

🌀 Methodology:

The project was developed using a structured approach:

3.1 Requirement Analysis:

Identified required features such as authentication, CRUD operations, and role control

Designed database schema based on project requirements

3.2 System Design

Backend: REST API using Express.js

Frontend: React (Vite + Tailwind CSS)

Database: MySQL

3.3 Implementation

Developed APIs for login, user profile, and data handling

Built UI components for interaction

Integrated authentication middleware

3.4 Testing

Tested API endpoints using frontend and Postman

Verified role restrictions and error handling

🌀 System Architecture:

The system follows a 3-tier architecture:

◆ Frontend (Client)

- Built using React
- Handles UI and user interactions

◆ Backend (Server)

- Node.js + Express
- Handles API requests and business logic

◆ Database

- MySQL
- Stores users, roles, and application data

🌀 **Functionality:**

5.1 Authentication System

Users log in using email and password

JWT-based authentication is used

Middleware verifies user identity

5.2 Role-Based Access Control (RBAC)

Admin:

- Full access (CRUD operations)
- Regular User:
- Limited or read-only access

5.3 CRUD Operations

Users can:

- Create records
- View records
- Update records
- Delete records

5.4 Member Portfolio / Dashboard

Displays user-specific data

Shows relevant database entries

5.5 Logging System

Tracks actions such as:

Login

Profile viewing

Data access

🌀 **Approach**

The approach followed in this project includes:

- Modular backend structure (routes, middleware, utils)
- Separation of concerns (frontend vs backend)
- Secure API design with authentication tokens
- Clean UI for better usability
- Database normalization to avoid redundancy

🌀 Implementation Details:

7.1 Backend

Express.js used to create REST APIs

Middleware for authentication and authorization

Logging utility for tracking actions

7.2 Frontend

React with Vite for fast development

Axios for API calls

Tailwind CSS for styling

7.3 Database

Tables include:

Member

Courses / Attendance (or your tables)

Relations defined using foreign keys

🌀 Team Members Contributions:

Sr. No.	Name	Roll No.	Justification
1.	Aryan Ashwin Meshram	24110054	Module B: API Development
2.	Jahan Zaib	24110140	Module A: B+ tree
3.	Mateen Sharief	24110197	Module B: SQL Indexing and optimization report
4.	Pallav Biyala	24110234	Module A: B+ tree
5.	Saksham Chourasia	24110321	Module B: API Development and UI Interface

🌀 Reference and external tools used:

1. ChatGPT
2. Copilot
3. GeekForGeeks



